QueryAPIGate HOW-TO GUIDE 34 · OPERATE

Upgrade to a new version safely

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=34-upgrade-safely

---

Time: 15 minutes. **You'll end up with:** a routine for upgrades - what to read first, what to back up, what to watch afterwards - and a way back if one goes wrong.

How versions work here

From the changelog's versioning policy:

- **A patch release** (`0.14.0` → `0.14.1`) never breaks anything covered: the REST API, CLI, environment variables, the store, grant fields, the MCP tool contract, error `code`s.
- **A minor release** (`0.14.x` → `0.15.0`) may - before 1.0 it's this project's equivalent of a major release - but only with a **Breaking** note in its changelog entry. Never by surprise.
- **Experimental** features may change in any minor release - the list is in the changelog's versioning policy.
- **Deprecated** things keep working until at least the next minor release (before 1.0) or the next major (from 1.0), and say so while you use them - see below.

Before: read, then back up

1. **Read the changelog** for every version between yours and the target: anything under **Breaking**, **Deprecated** and **Upgrading**. Note any Python version change - 0.14 requires Python 3.11 or newer. And whether it's a rolling upgrade: from 0.37 to 0.38 it isn't - stop every instance first.
2. **Back up the store** - Back up and restore. This is your way back: going to an older version in place is not supported.
3. **Pin the target version** - never upgrade by accident:

```shell
pip install "queryapigate[server,postgres]==0.48.0"      # not just "queryapigate"
```

```yaml
image: ghcr.io/anantharajuc/queryapigate:0.48.0           # not :latest
```

During: just start it

```shell
docker compose pull && docker compose up -d       # or: pip install ... && systemctl restart
queryapigate
```

The store is upgraded automatically on first start - nothing to run by hand. Verified across releases: a `/data` volume written by **0.7.0** (connections and API keys still in JSON files) started under 0.14 with its connection and key intact, and the key authenticated with its original secret. **Since 0.15, a home last run by 0.10 or older must go through 0.14 first** - 0.15 no longer reads those JSON files, and refuses to start on such a home, saying so.

If the store can't be used safely, the server **refuses to start**, with a message saying what's wrong and what to do, before changing anything - for example, a store a *newer* release has already upgraded. It never starts and then fails on first use.

After: check the log, then the deprecation warnings

```shell
curl https://api.example.com/health                # the new "version"
docker compose logs queryapigate | grep -i warn      # or journalctl -u queryapigate
```

Look for two kinds of warning.

Something you configured is deprecated. For example, 0.14 logged this for a home still holding the pre-SQLite JSON files:

```text
WARNING queryapigate: Importing the pre-SQLite JSON files (db_connections.json, saved_sql/,
api_keys.json, roles.json,
audit_log.json) on first start is deprecated since 0.14.0 and may be removed in 0.15.0: ...
```

- and 0.15 removed it. Each deprecation warning names what to use instead and the earliest release that may remove it; act on it before that release.

A caller uses something deprecated. The first call to a deprecated route in each process is logged with the key that made it - 0.14, for the routes 0.15 then removed:

```text
WARNING queryapigate [5348a82deb98 key=app-alice]: /execute_sql_from_file is deprecated since 0.14.0 and
may be
removed in 0.15.0: use /q/{name} instead ...
```

The response itself carries a `Deprecation` header (RFC 9745) and `Link: </q/{name}>; rel="successor-version"`, and `/openapi.json` marks the operation `deprecated: true` - so a client's own tooling can notice. To find **every** key still calling it, ask `/metrics`, which counts requests by endpoint and key:

```shell
curl -s https://api.example.com/metrics | grep 'endpoint="api.execute_sql_from_file"' | grep
requests_total
# queryapigate_requests_total{method="POST",endpoint="api.execute_sql_from_file",status="400",key="app-
alice"} 1
```

Move those callers before upgrading to the release that removes it. The current list of deprecations is the **Deprecated** sections of the changelog.

Since 0.16, a `break_glass_used` warning means the shared `QUERYAPIGATE_API_KEY` was used although named owners exist - expected while you move people to their own tokens (guide 43), worth a look afterwards.

Finally, glance at **Alerts** in the Console for anything the upgrade surfaced - a connection failing because a driver went missing, say.

Going back

There's no in-place downgrade: an older release refuses a store a newer one has upgraded. To roll back, stop the server, restore the backup you took before upgrading, and start the old version. Runs and changes made since the backup are lost - which is why the backup comes right before the upgrade, not the night before.

Several instances, one store

With a shared PostgreSQL store, upgrade them together: stop all, upgrade one and let it migrate the store, then start the rest on the same version. Don't run old and new versions against the same store at once.

Next steps

- Back up and restore the store.
- Deploy with Docker for real.
- The changelog.